

Real-Time & Multi-User State Cheatsheet

Lessons: [58](#) — [WebSockets & SSE](#) · [67](#) — [Multi-User State in One Process](#)

Examples: [58](#) · [67](#)

Covers: SSE, the WebSocket wire format, the hub, backpressure, state scopes, per-user maps, scaling out

Legend: [*] = API the lessons have not covered yet

SERVER-SENT EVENTS (SSE)

```
one-way                server -> browser, over plain HTTP/1.1
w.Header().Set("Content-Type", "text/event-stream")
w.Header().Set("Cache-Control", "no-cache")
w.Header().Set("Connection", "keep-alive")
the wire format        field: value pairs, a BLANK LINE ends the event
  id: 42
  event: message        the client's addEventListener name (default "message")
  data: {"text":"hi"}    repeat data: for multi-line payloads
  retry: 3000           tell the client how long to wait before reconnecting
  (blank line)
flush after every event w.(http.Flusher).Flush() – or nothing is ever sent
                        (or http.NewResponseController(w).Flush())
disconnect             <-r.Context().Done() – the ONLY reliable signal
Last-Event-ID header   sent on reconnect; resume from that id
heartbeat              a ": ping\n\n" comment every ~15s keeps proxies open
WriteTimeout must be 0 or the server kills your stream mid-flight
browser auto-reconnects for free – the reason to prefer SSE for feeds
(SSE is text-only and one-way; that's usually all a notification feed needs)
```

WEBSOCKETS: the handshake

```
client sends           GET /ws with
  Upgrade: websocket
  Connection: Upgrade
  Sec-WebSocket-Key: <16 random bytes, base64>
  Sec-WebSocket-Version: 13
server replies 101     Switching Protocols with
  Sec-WebSocket-Accept: base64(sha1(key + "258EAF5-E914-47DA-95CA-C5AB0DC85B11"))
                        the magic GUID is from RFC 6455
hijack the connection  conn, buf, err := w.(http.Hijacker).Hijack()
                        – you now own the raw TCP socket; net/http is done
check the Origin header the browser will connect from ANY site otherwise
                        (there is no same-origin policy for WebSockets)
```

WEBSOCKETS: the frame

```
byte 0      FIN(1) RSV(3) OPCODE(4)
byte 1      MASK(1) PAYLOAD-LEN(7)
len 0-125   that's the length
len == 126  the next 2 bytes are the length (big-endian)
len == 127  the next 8 bytes are the length
mask key    4 bytes, present when MASK is set
payload     XOR each byte with maskKey[i%4]
CLIENT -> SERVER MUST be masked; server -> client MUST NOT be
opcodes
  0x0 continuation      a fragmented message
  0x1 text (UTF-8)
  0x2 binary
  0x8 close              optional 2-byte status code + reason
  0x9 ping               -> you MUST reply with pong
  0xA pong
control frames           <= 125 bytes, never fragmented, can arrive mid-message
```

THE HUB (one goroutine owns the state)

```
type Hub struct {
  clients    map[*Client]bool      owned by run(), never touched elsewhere
  register   chan *Client
  unregister chan *Client
  broadcast  chan []byte
}
func (h *Hub) run() {
  for {
    select {
    case c := <-h.register:  h.clients[c] = true
    case c := <-h.unregister: delete(h.clients, c); close(c.send)
    case msg := <-h.broadcast:
      for c := range h.clients {
        select {
        case c.send <- msg:      buffered, so a fast path
        default:                SLOW CLIENT: drop it rather than block
          delete(h.clients, c); close(c.send)
        }
      }
    }
  }
}
NO MUTEX ANYWHERE      the map has exactly one owner – that IS the lock
one writer goroutine per connection    concurrent Writes to a socket interleave
                                         and corrupt frames
one reader goroutine per connection
rooms                  map[RoomID]map[*Client]bool, same ownership rule
```

BACKPRESSURE & LIFECYCLE

buffered send channel	make(chan []byte, 256) per client
default: in the select	the drop policy – never block the hub on one slow client
drop vs disconnect	drop frames for a feed; disconnect for a chat
heartbeat	ping every 30s; if no pong within the deadline, close
read deadline	conn.SetReadDeadline, refreshed on every pong
message size limit	per connection, enforced before allocating
per-connection rate limit	messages/second, so one client can't flood the hub
close handshake	send a close frame, then close the socket
graceful shutdown	close the hub -> broadcast a close frame to everyone -> wait briefly -> close the listener

MESSAGE PROTOCOL

a JSON envelope	{"type":"chat.message","id":"...","data":{...}}
type-first	so the client switches on one field
version it	from the first day
validate everything	it's user input arriving on a long-lived connection
ack ids for delivery	if the client needs to know it arrived
(design the protocol before the transport – the transport is the easy half)	

STATE SCOPES (the PHP contrast)

PHP/FPM	a process per request; nothing survives it. State goes to Redis or the DB because it has nowhere else to live.
Go	ONE process, ONE goroutine per request, and the process OUTLIVES every request. Package-level state is shared, concurrently, by every request. That's the whole shift.
request scope	the local variables and r.Context(); dies with the response
user scope	map[UserID]*State, guarded – lives across requests
process scope	package vars, caches, hubs, pools – lives until restart
cluster scope	Redis/Postgres – the only scope that survives 2 replicas
(pick the smallest scope that works; every step up costs correctness)	

PER-USER STATE IN ONE PROCESS

type Store struct { mu sync.RWMutex; m map[UserID]*State }	
RLock for reads / Lock for writes	and NEVER return the inner pointer without a plan for who locks it
the shard fix	N maps + N mutexes, keyed by hash(id)%N
per-user rate limiter	map[UserID]*rate.Limiter – plus eviction (see lesson 68)
presence	map[UserID]time.Time of last seen
sweeper goroutine	a ticker that evicts anything older than the TTL – without it, every map in this list is a memory leak
identity in ctx	set once by auth middleware; read everywhere downstream

THE RACES YOU WILL HIT

check-then-act	<code>if !exists(k) { create(k) }</code> – two goroutines both pass the check. Fix: hold ONE lock across both steps, or <code>LoadOrStore</code> , or a unique constraint in the database.
lost update	read 10, add 1, write 11 – twice, and you have 11. Fix: <code>atomic.Int64</code> , a lock around read-modify-write, or <code>UPDATE ... SET n = n + 1</code> in SQL.
double submit	the double-clicked "Pay" button: two identical requests arrive in parallel, both pass every check. Fix: an idempotency key + a <code>UNIQUE</code> constraint, so the database rejects the second one. A mutex only works while you have exactly one replica.
map + concurrent write	the runtime <code>PANICS</code> ("concurrent map writes"). It is not a silent corruption – it kills the process.
the <code>r.Context()</code> trap	<code>go func(){ useCtx(r.Context()) }()</code> – the context is cancelled the moment the handler returns, so the background work dies instantly. Fix: <code>context.WithoutCancel(ctx)</code> , or a fresh context with its own timeout.
goroutine per request leak	a background goroutine with no exit path, once per request

WHAT BREAKS AT TWO REPLICAS

in-process hub	replica A's broadcast never reaches replica B's clients
in-memory sessions	a user hitting the other replica is logged out
per-process rate limits	the effective limit is $N \times$ what you configured
local caches	N different answers, none of them wrong-looking
sticky sessions	a workaround, not a fix; it breaks on deploy
THE FIX	a pub/sub backplane behind an interface:
<pre>type Backplane interface { Publish(ctx context.Context, room string, msg []byte) error Subscribe(room string) <-chan []byte }</pre>	
each replica subscribes; a local broadcast also publishes; every replica delivers to ITS OWN clients. Redis pub/sub, NATS, or Postgres LISTEN/NOTIFY.	
sessions/limits/state	move to Redis at the same time
(write it behind an interface from day one – the single-process version is the fake)	

TRAPS & MEMORIZE

no Flush on SSE	the client receives nothing, and you debug for an hour
<code>WriteTimeout</code> set on a stream	the server cuts your long-lived connection
<code>conn.Write</code>	interleaved frames; one writer goroutine per connection
no Origin check on /ws	any website can open a socket as your logged-in user
unbounded send channel	a slow client becomes unbounded memory
blocking the hub	one slow client freezes every other client
map without a mutex	a runtime panic under real traffic
returning the inner pointer	the lock protected the map, not the value
no sweeper	presence/limiter/session maps grow forever
<code>r.Context()</code> for background work	cancelled at response time
assuming one replica	everything above works perfectly until you scale to 2
testing with one client	every one of these bugs needs two